ENTERPRISE CHALLENGE | ATIVIDADE 5

Smart HAS - AI Logistics Extension

Relatório final de projeto

Rede de vizinhança que devolve ao uso as ferramentas e os materiais parados, com inteligência artificial e regras de negócio no banco Oracle

Integrante: Yuri Raphael Alves de Sousa
RM: 556163 | Entrega individual
Empresa parceira do desafio: Leroy Merlin
FIAP - Faculdade de Informática e Administração Paulista
Outubro de 2026

Repositório com o código-fonte e a documentação:
https://github.com/YuriRASousa/ComunidadeConectada_FASE5

Sumário

1. Introdução

1.1 Fundamentação teórica

1.2 Storytelling e ideação

2. Metodologia

2.1 Tecnologias e ferramentas

2.2 Arquitetura e casos de uso

3. Discussão

3.1 Perspectiva com mais tempo

3.2 Estimativa de custo

4. Lições aprendidas

5. Conclusão

Anexo A - Documentação e código-fonte

1. Introdução

O Smart HAS (Comunidade Conectada) é uma plataforma de vizinhança em que moradores de uma mesma região publicam o que têm parado em casa e encontram o que precisam por perto, por empréstimo, troca ou doação. No desafio proposto pela Leroy Merlin, o produto foi enquadrado como a AI Logistics Extension: uma camada de inteligência artificial e de logística de proximidade que aproxima quem tem de quem precisa, no ramo de ferramentas e materiais de construção e reforma.

Objetivos e escopo

- Permitir que um morador publique um item com categoria, estado de conservação, tipo (empréstimo, troca ou doação) e localização.
- Listar os itens ordenados pela distância até quem procura, para que a retirada seja feita a pé, sem frota e sem centro de distribuição.
- Controlar o ciclo do empréstimo (pedido, uso, devolução, atraso) com as regras de negócio dentro do banco Oracle, em pacotes PL/SQL.
- Oferecer assistência de IA generativa: descrever o item a partir de um texto curto e responder dúvidas de uso e segurança (ConectaIA, com Gemini).
- Dar à operação um painel administrativo com usuários, itens, empréstimos em aberto, alertas de atraso e rotinas do banco.
- Entregar o sistema por um pipeline de CI/CD que compila, testa e publica a cada alteração.

Ficam fora do escopo desta entrega: pagamento, logística com transportadora, notificações push e a publicação nas lojas de aplicativos. Esses pontos aparecem na seção 3.1, como evolução.

O problema real

Uma furadeira de uso doméstico trabalha, em média, poucos minutos em toda a sua vida útil, mas é comprada por praticamente todo domicílio que precisa pendurar uma prateleira. O mesmo vale para escada, serra, lixadeira, projetor, panela de pressão e as sobras de tinta, piso e madeira de uma obra. De um lado há recurso parado, do outro há compra por impulso para um uso pontual, e o vizinho que precisa não sabe quem tem a poucas quadras de distância. O que falta entre as duas pontas é um catálogo local, a distância e a confiança para emprestar.

Para a empresa parceira, o problema tem duas faces. Para o cliente, uma obra pequena exige ferramentas caras que serão usadas uma vez. Para o negócio, o varejo de materiais ganha ao ser o ponto de partida de projetos de reforma, e perde cliente que desiste da obra porque o custo do conjunto de ferramentas pesa. Uma rede de empréstimo entre vizinhos reduz essa barreira e mantém o morador dentro do ecossistema de reforma.

1.1 Fundamentação teórica

O tema central do desafio é o uso inteligente de recursos por meio de tecnologia: consumo colaborativo, logística de proximidade e inteligência artificial como mediadora. Quatro fundamentos sustentam as decisões do projeto.

Consumo colaborativo e economia compartilhada. Botsman e Rogers (2010), em What's Mine Is Yours, descrevem o deslocamento do valor da posse para o acesso: o que importa é poder usar o bem quando necessário, não ser dono dele. Plataformas que coordenam oferta e demanda ociosas funcionam quando resolvem a confiança entre desconhecidos. No Smart HAS, a confiança aparece na reputação calculada por avaliações, no registro de cada empréstimo e na auditoria das mudanças.

Economia circular. A Fundação Ellen MacArthur define a economia circular como um modelo que mantém produtos e materiais em uso pelo maior tempo possível. Emprestar uma ferramenta adia uma compra e o descarte associado a ela. É o encaixe do projeto com a agenda de sustentabilidade de uma varejista de materiais.

Sociedade 5.0. Conceito apresentado pelo governo do Japão (Cabinet Office, 2016) para uma sociedade que integra espaço físico e espaço virtual para resolver problemas sociais, com a tecnologia a serviço das pessoas. O Smart HAS conecta pessoas de uma comunidade por meio de dados de localização, catálogo e assistente de IA, que é a premissa da disciplina.

IA generativa como camada de mediação. Modelos de linguagem de grande porte, como o Gemini, produzem texto a partir de instruções curtas. No projeto eles resolvem duas fricções concretas: o anúncio mal escrito, que ninguém encontra, e a dúvida de quem nunca usou a ferramenta. A decisão de uso foi deliberadamente conservadora: a IA sugere e explica, mas a regra de negócio (quem pode emprestar, por quantos dias, quando há atraso) fica em código determinístico e testado, não em um modelo probabilístico.

Regra de negócio no banco e entrega contínua. Do lado da engenharia, duas práticas guiaram a arquitetura: manter as invariantes do negócio perto dos dados (pacotes PL/SQL, triggers e restrições), de modo que nenhum cliente consiga contorná-las, e automatizar integração e entrega, conforme Humble e Farley (2010) em Continuous Delivery, para que cada mudança seja verificada do mesmo jeito, sempre.

A conexão entre teoria e prática é direta: o conceito de acesso sobre posse virou o catálogo e o empréstimo; a confiança virou reputação e auditoria; a economia circular virou o argumento de valor para a parceira; a IA entrou onde reduz atrito sem assumir decisões críticas.

1.2 Storytelling e ideação

A história começa com uma cena comum. Marina precisa furar uma parede para instalar uma prateleira e não tem furadeira. Comprar custa mais do que o valor do serviço de um profissional, e ela usaria o aparelho uma vez. Dois andares acima, Leticia tem uma furadeira de impacto guardada há meses. As duas moram a algumas quadras e nunca conversaram sobre isso.

A ideia que resolveu esse encontro foi tratar o que já existe na vizinhança como estoque disponível. A pergunta de projeto deixou de ser como vender mais e passou a ser como fazer o que está parado voltar a circular, com segurança para quem empresta. Entre as alternativas consideradas (um mural de classificados, um marketplace de aluguel pago e uma rede de empréstimo gratuita com reputação), a terceira foi escolhida por três motivos: não depende de pagamento, o que reduz o risco legal e o atrito de entrada; tem o menor custo de operação; e combina com a mensagem de sustentabilidade da empresa parceira.

O impacto esperado se divide em três frentes. Para o morador, economia por evitar compras pontuais. Para a comunidade, mais giro de itens e vínculo entre vizinhos. Para a empresa parceira, a oportunidade de se posicionar como o ponto de partida de projetos de reforma que o morador agora consegue iniciar com menos custo.

A evolução do protótipo aconteceu em camadas, cada uma resolvendo uma limitação da anterior:

Etapa	O que existia	Limitação que motivou o passo seguinte
Fases 1 a 4	Protótipo em Flutter com dados simulados em memória	Sem persistência, sem autenticação e sem backend
Fase 5 (14/08 a 29/08)	App em React Native (10 telas), backend Spring Boot com Firebase, painel Angular, ConectaIA com Gemini	Regras de negócio no código da aplicação e dados em um banco não relacional
Fase 6 (30/08 a 04/10)	Migração para Oracle: 11 tabelas, functions e procedures PL/SQL, cargas e consultas	Rotinas soltas, sem trilha de auditoria e sem entrega automatizada
Atividade 4 (setembro)	Pitch no formato do Enterprise Challenge, com a AI Logistics Extension	Apresentação do valor, ainda sem a camada de entrega
Fase 7 (04/10 a 31/10)	4 pacotes PL/SQL, 3 triggers, SQL dinâmico, 15 testes e pipeline no GitHub Actions	Evolução para produção: ver seção 3.1

Quadro 1 - Evolução do projeto, da primeira versão ao protótipo funcional

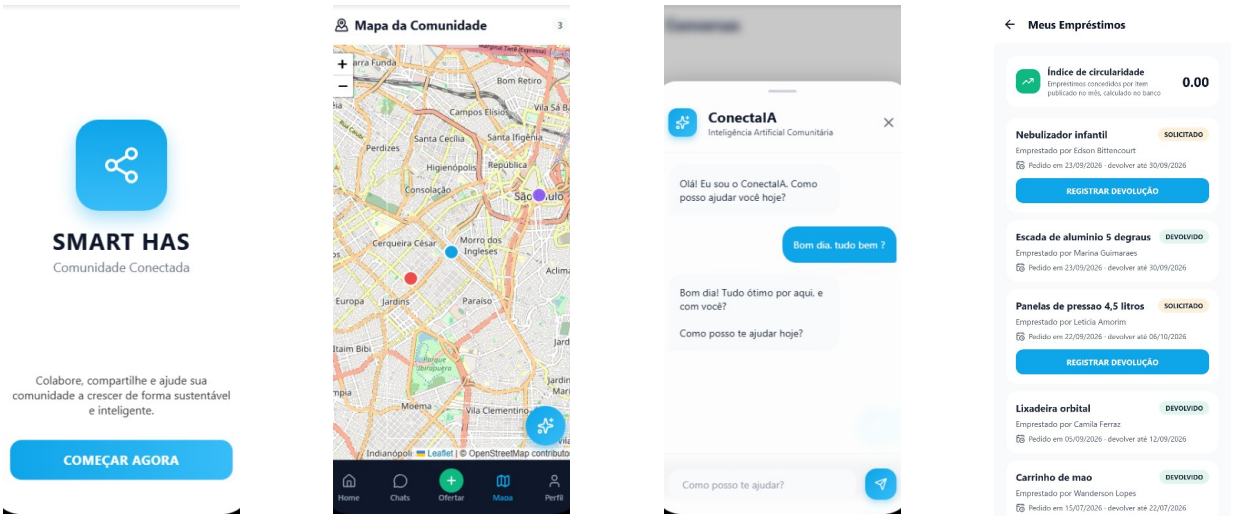


Figura 1 - O protótipo funcional: catálogo, mapa da comunidade, assistente ConectaIA e Meus empréstimos

2. Metodologia

O trabalho foi conduzido de forma iterativa e incremental, com fluxo no estilo Kanban: um item em andamento por vez, puxado de uma lista priorizada, sempre terminando em algo executável antes de abrir o próximo. A cadência foi dada pelas entregas da disciplina (Fase 5, Fase 6, Atividade 4, Fase 7 e este relatório), que funcionaram como as sprints do projeto, cada uma com um objetivo claro e um critério de aceite verificável. Por ser uma entrega individual, não foram simuladas cerimônias de Scrum que dependem de uma equipe; a revisão de cada ciclo foi feita por meio de testes, demonstrações com dados reais e inspeção do resultado.

As fases do projeto e o que foi feito em cada uma:

Fase	Atividades	Como foi validado
Planejamento	Definição do problema, do público e do escopo; escolha da stack alinhada ao material da disciplina; desenho das telas e do contrato da API (API_CONTRACT.md)	Revisão do contrato antes de codificar; as 10 telas definidas junto com os endpoints
Desenvolvimento	App React Native, backend Spring Boot, painel Angular, ConectaIA; depois a migração para Oracle e as regras em PL/SQL	Execução contra a API real a cada tela; migração conferida com consultas de contagem antes e depois
Testes	15 testes automatizados (14 de integração no Oracle e 1 de inicialização); testes de fumaça da API empacotada	Execução local e no GitHub Actions; relatórios Surefire como artefato
Validação	Demonstração com dados simulados (seed) de moradores, itens e empréstimos, incluindo um atraso proposital; tentativas de injeção de SQL	Banco recusa o que deve recusar (ORA-20301, 20401 e outros); painel mostra atrasos e alertas
Entrega	Pipeline CI/CD, imagem Docker, relatórios e vídeos de demonstração	Execuções verdes na branch main, com 4 jobs

Quadro 2 - Fases do projeto e formas de validação

As hipóteses de produto foram tratadas como perguntas a verificar no próprio protótipo. A hipótese de que a lista ordenada por distância tornaria a busca útil foi verificada com coordenadas de um bairro de São Paulo na carga de dados e com a função de distância (haversine) do pacote PKG_SH_CALCULOS, coberta por teste automatizado. A hipótese de que o banco poderia garantir as regras sozinho foi testada tentando violá-las: reabrir um empréstimo devolvido, emprestar um item a seu próprio dono, informar uma tabela inexistente. Em todos os casos a recusa vinha do Oracle, e não do cliente.

A figura abaixo situa as entregas no tempo, a partir do histórico real do repositório.

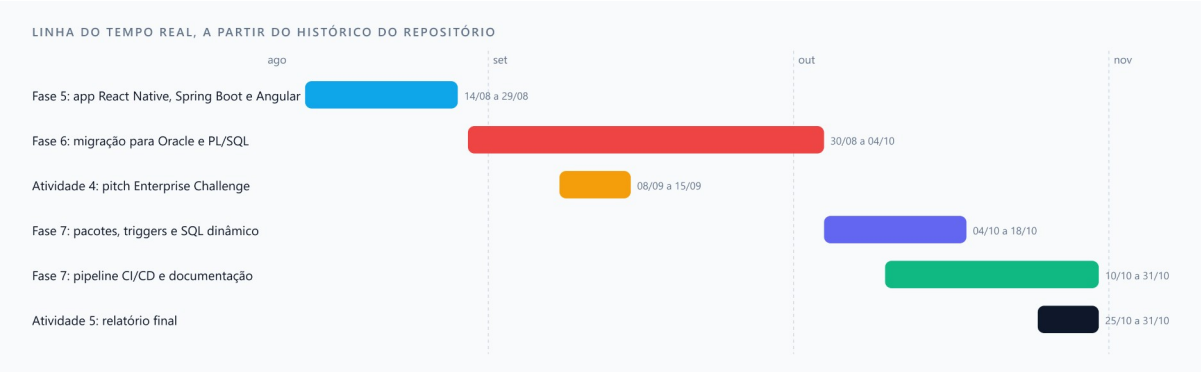


Figura 2 - Linha do tempo das fases e das atividades

2.1 Tecnologias e ferramentas

Categoria	Tecnologias e ferramentas
Desenvolvimento	TypeScript; Java 21; React Native 0.86 com Expo 57 e React Navigation; Spring Boot 4.1 (Spring Web MVC, Spring Security, Validation, JDBC); Angular 20; JWT (jjwt 0.12); OpenAPI/Swagger (springdoc 2.8); Maven; npm; Leaflet com OpenStreetMap no mapa
Banco de dados	Oracle Database 19c (servidor acadêmico da FIAP) e Oracle Free 23ai nos testes do pipeline; PL/SQL com pacotes, triggers, SQL dinâmico, BULK COLLECT e FORALL; driver ojdbc11; Firebase (Authentication e Firestore) na Fase 5, depois substituído
IA e ML	Gemini, por meio do Google AI Studio (API de IA generativa), para o botão Melhorar com IA e o assistente ConectaIA. Não há modelo treinado no projeto: o uso é de inferência por API, com a chave fora do código-fonte
Infraestrutura	Docker (imagem do backend e docker-compose); GitHub Actions como pipeline; GitHub Container Registry; Oracle Free como serviço de teste no pipeline; scripts em Python (python-oracledb) para instalar o schema; Swagger como painel de exploração da API
Gestão	Git e GitHub (versionamento e histórico); fluxo Kanban simplificado; contrato de API em Markdown; documentação gerada por scripts Node (pptxgenjs, docx, sharp) e convertida para PDF com LibreOffice; vault de notas em Obsidian

Quadro 3 - Tecnologias e ferramentas por categoria

2.2 Arquitetura e casos de uso

A solução tem três clientes e uma única API. O aplicativo do morador e o painel administrativo conversam com o backend Spring Boot por HTTPS, autenticados com JWT. O backend é o único componente que acessa o Oracle (por JDBC) e chama os pacotes PL/SQL. O assistente de IA é chamado pelo aplicativo, com a chave configurada por variável de ambiente. O pipeline no GitHub Actions compila, testa e publica a cada alteração.

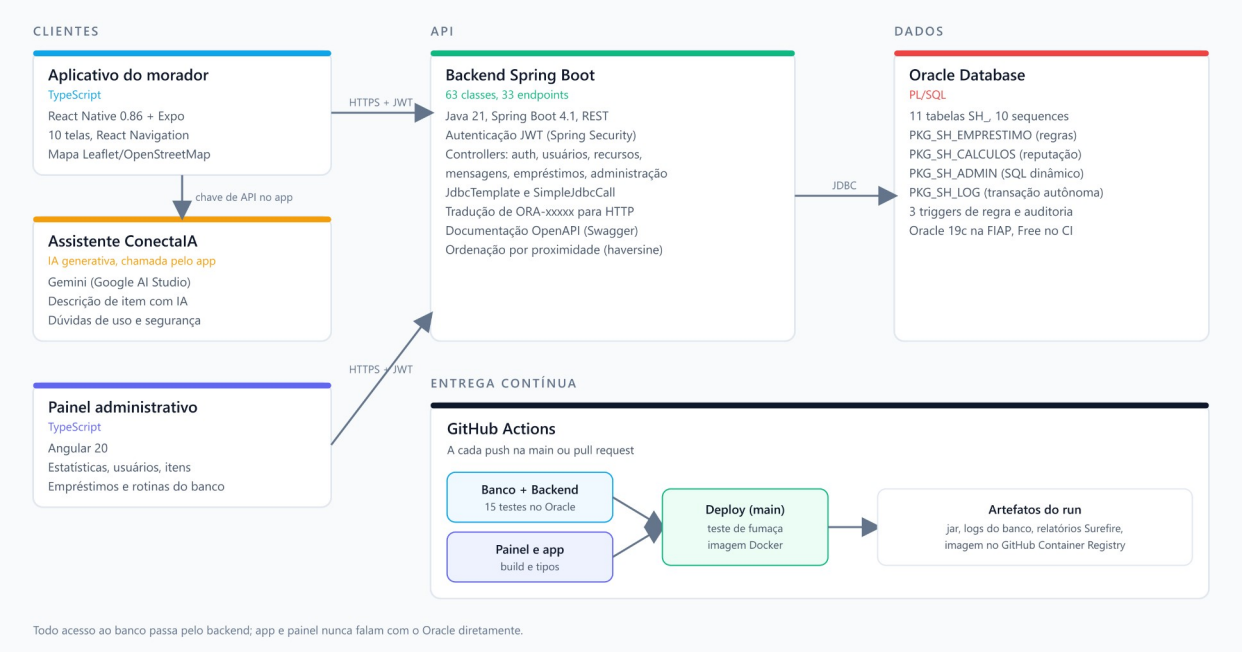


Figura 3 - Arquitetura geral da solução

O fluxo principal do usuário, do anúncio à devolução, e como o painel acompanha, está resumido abaixo.



Figura 4 - Fluxo do empréstimo e as rotinas do banco em cada passo

Caso de uso	Tecnologias aplicadas	Onde mora a regra
Autenticação e papéis	Spring Security, JWT, tela de login no app e no painel	Backend; a trigger TRG_SH_USUARIO_AUD audita mudança de papel e nunca toca na senha
Persistência de dados	Oracle, JdbcTemplate, 11 tabelas, 10 sequences, índices	Banco (restrições e pacotes)
Publicar e encontrar itens	React Native, endpoint de recursos, haversine, mapa Leaflet	PKG_SH_CALCULOS.DISTANCIA_KM e busca ordenada por proximidade
Pedir e devolver	Endpoints de empréstimo, SimpleJdbcCall	PKG_SH_EMPRESTIMO (prazo de 7 dias, validações, reputação)
Atrasos e alertas	Painel Angular, rotina de detecção	DETECTA_ATRASOS: severidade MEDIA a partir de 5 dias e ALTA a partir de 15
Inteligência artificial	Gemini (Google AI Studio), tela de ofertar e chat	Sugestão de texto e resposta a dúvidas; decisões de negócio ficam no banco
Auditoria e rastreabilidade	Triggers, tabela SH_AUDITORIA, PKG_SH_LOG com transação autônoma	Banco; o erro fica registrado mesmo quando a operação sofre rollback
Consultas administrativas	SQL dinâmico com bind e lista fechada, DBMS_ASSERT	PKG_SH_ADMIN (volumetria, contagem e busca com filtros)
Dashboards	Angular, endpoints de administração	Resumo mensal por BULK COLLECT e FORALL
Entrega contínua	GitHub Actions, Maven, Docker, GHCR	Pipeline versionado em .github/workflows/ci-cd.yml

Quadro 4 - Casos de uso e as tecnologias aplicadas em cada um

3. Discussão

Desempenho em relação ao cronograma e ao orçamento

O cronograma seguiu a sequência das entregas da disciplina: Fase 5 entre 14 e 29 de agosto, migração para o Oracle entre o fim de agosto e o início de outubro, o pitch do Enterprise Challenge em setembro, a Fase 7 em outubro e este relatório no fim do mês, como mostra a Figura 2. Todas as entregas chegaram ao protótipo funcional previsto, com o código no repositório e a demonstração gravada.

Sobre o orçamento, é preciso ser transparente: não houve um orçamento formal aprovado no início do projeto. O desembolso real foi nulo, porque todo o desenvolvimento usou camadas gratuitas e o servidor Oracle acadêmico da FIAP, e o trabalho foi feito por uma única pessoa. A estimativa da seção 3.2 foi construída agora, a partir do que o sistema contém, e funciona como a linha de base de custo para uma versão de produção.

Ajustes que foram necessários ao longo do caminho:

- Troca de stack mobile: de Flutter para React Native, para alinhar o projeto ao conteúdo da disciplina e testar contra a API real.
- Troca de banco: do Firebase para o Oracle, para levar as regras de negócio ao banco e trabalhar com PL/SQL; exigiu reescrever a camada de persistência do backend e uma carga de dados nova.
- Limitação de ambiente: o emulador Android travou por conflito com um driver anti-cheat, e a validação do app foi feita pela versão web (react-native-web) contra o backend real. Câmera e notificações push ficaram sem teste em dispositivo nativo.
- Mapa: o uso do Leaflet com OpenStreetMap evitou uma chave paga do Google Maps.
- Rotinas soltas viraram pacotes na Fase 7, com migração em dois passos (criar, testar e só então remover as antigas).

Maturidade do protótipo

O protótipo está no nível de demonstração funcional validada em ambiente de teste: o ciclo completo funciona de ponta a ponta contra um Oracle real, com 15 testes automatizados passando em todas as execuções do pipeline e a API empacotada passando por teste de fumaça. Isso é mais do que uma maquete, mas ainda não é produção. Os dados são simulados, não há usuários reais, o aplicativo não foi exercitado em aparelho físico e a IA depende de uma chave pessoal de desenvolvimento. Em uma escala de maturidade tecnológica, o estágio é o de protótipo validado em laboratório.

3.1 Perspectiva com mais tempo

Frente	O que seria feito
Técnica	Testar em aparelhos Android e iOS; notificações push (lembrar a devolução e avisar o pedido); upload de fotos de itens com armazenamento de objetos; sessão persistente no app; reconciliar o modelo de conversas com a tela de chat por item
Escalabilidade	Banco gerenciado (Autonomous Database na OCI), pool de conexões dimensionado, cache de buscas, execução do backend em container orquestrado, observabilidade com métricas e alertas
Funcionalidades	Painel Angular com as telas de volumetria e de busca dinâmica (hoje só na API), reservas com antecedência, grupos por condomínio, avaliação mútua, doação e troca com fluxo próprio
IA	Recomendação do item mais adequado ao projeto do morador (por exemplo, listar as ferramentas de uma reforma de banheiro), reconhecimento de item por foto, moderação automática de anúncios e resposta a dúvidas com base no catálogo da parceira
Integração com a parceira	Ligar o catálogo da Leroy Merlin: sugerir a compra do material que não existe na vizinhança, ofertar pontos de retirada em loja e medir quanto de projeto de reforma foi iniciado por empréstimo
Segurança e conformidade	Adequação à LGPD (consentimento de localização, anonimização), verificação de identidade do morador, política de responsabilidade em caso de dano ao item
Interface	Teste de usabilidade com moradores reais, acessibilidade, modo escuro e versão em outros idiomas

Quadro 5 - Evolução prevista com mais tempo de desenvolvimento

3.2 Estimativa de custo

A estimativa responde à pergunta: quanto custaria construir este mesmo sistema como projeto profissional, com uma equipe contratada? Os valores por hora partem da Pesquisa Salarial APinfo 2025 (coleta de agosto a setembro de 2025), usando o rendimento mensal PJ de cada cargo dividido por 160 horas. Onde a pesquisa não traz o cargo exato, a linha mais próxima foi usada como referência e está indicada.

Mão de obra:

Função	Referência APinfo 2025	PJ mensal	Valor/hora	Horas	Subtotal
Gerente de projetos	Gerente de projetos (PMO)	R$ 16.703	R$ 104,39	60	R$ 6.263,63
Desenvolvedor mobile e painel	Desenvolvedor Front-End	R$ 8.776	R$ 54,85	150	R$ 8.227,50
Desenvolvedor back-end	Desenvolvedor Back-End	R$ 9.865	R$ 61,66	170	R$ 10.481,56
Analista Oracle PL/SQL	Analista de sistemas Oracle PL/SQL	R$ 10.469	R$ 65,43	70	R$ 4.580,19
DBA	DBA (administração de base)	R$ 11.118	R$ 69,49	50	R$ 3.474,38
DevOps	Desenvolvedor Full-Stack (referência)	R$ 13.701	R$ 85,63	50	R$ 4.281,56
QA (analista de testes)	QA (analista de testes)	R$ 8.270	R$ 51,69	60	R$ 3.101,25
UX/UI designer	Web Designer (referência)	R$ 5.138	R$ 32,11	70	R$ 2.247,88
Total de mão de obra				**680**	**R$ 42.657,94**

Quadro 6 - Horas por função e valor médio da hora (fonte: APinfo, Pesquisa Salarial 2025)

Distribuição de horas: back-end e mobile concentram a maior parte (320 de 680 horas), seguidos pelo banco (DBA e PL/SQL, 120 horas), UX, QA, DevOps e gestão. As horas são uma estimativa de esforço do que foi construído (63 classes Java, cerca de 4 mil linhas de TypeScript, 11 scripts SQL com quase 2 mil linhas e o pipeline), e não um registro de horas trabalhadas.

Infraestrutura e licenças em produção (primeiro ano):

Item	Período	Valor unitário	Total no ano
Banco Oracle gerenciado (Autonomous Database)	12 meses	R$ 900 / mês	R$ 10.800
Hospedagem do backend (container)	12 meses	R$ 150 / mês	R$ 1.800
API de IA generativa (Gemini), uso moderado	12 meses	R$ 100 / mês	R$ 1.200
Conta de desenvolvedor Apple (US$ 99/ano)	1 ano	R$ 550	R$ 550
Conta de desenvolvedor Google Play (US$ 25, única)	1 vez	R$ 140	R$ 140
Domínio do painel e do app	1 ano	R$ 50	R$ 50
Total de infraestrutura e licenças			**R$ 14.540**

Quadro 7 - Custos de infraestrutura estimados para o primeiro ano de operação

Os valores do Quadro 7 são estimativas arredondadas de referência, em reais, e devem ser confirmados na calculadora de cada provedor antes de uma contratação. Durante o desenvolvimento o custo foi zero: Oracle acadêmico da FIAP, camada gratuita do GitHub (Actions e Container Registry), Expo e uma chave de desenvolvimento do Gemini.

Resumo do custo total estimado:

Componente	Valor
Mão de obra (680 horas)	R$ 42.657,94
Reserva de contingência (10% da mão de obra)	R$ 4.265,79
Infraestrutura e licenças (12 meses)	R$ 14.540,00
Custo total estimado	**R$ 61.463,73**

Quadro 8 - Resumo do custo total estimado do projeto

Para efeito de comparação, o desembolso efetivo do protótipo foi de R$ 0,00, porque o trabalho foi individual e usou recursos acadêmicos e gratuitos. A diferença entre os dois números é o valor do trabalho que uma empresa precisaria remunerar para obter o mesmo resultado.

4. Lições aprendidas

Desafios técnicos e de organização que foram superados

Desafio	O que aconteceu	Como foi resolvido
Emulador Android	Travou por conflito com um driver anti-cheat, e reiniciar a máquina só para isso não fazia sentido	Validação pela versão web do app contra o backend real; registro explícito do que ficou sem teste nativo
Migração Firebase para Oracle	Mudança de modelo de dados (documentos para tabelas) com o app já funcionando	Esquema novo, carga de dados e testes de integração antes de trocar a persistência
Regra de negócio espalhada	Funções soltas, valores fixos no código e nenhuma trilha de auditoria	Pacotes com constantes, triggers de auditoria e erros de aplicação traduzidos para HTTP
SQL dinâmico e injeção	Concatenar nome de tabela ou ordenação abre brecha	Valores por bind, tabelas validadas no dicionário (USER_TABLES, DBMS_ASSERT) e ordenação em lista fechada, com testes de ataque
Pipeline com Oracle	O banco precisa existir e estar saudável dentro do job	Oracle Free como serviço do GitHub Actions, com healthcheck e o mesmo script de instalação usado localmente
Geração de documentos	A conversão de slides para PDF falhou com cartões altos sobrepostos e sombra	Sombra desligada nesses cartões; verificação visual de cada página antes de entregar
Limpeza de dados de teste	Uma limpeza ampla apagou os alertas antigos do banco compartilhado	Reconhecido e documentado; a rotina de detectar atrasos recria os alertas, e as limpezas passaram a filtrar por identificador

Quadro 9 - Desafios técnicos e soluções

O que funcionou bem

- Entregas pequenas e verificáveis: cada ciclo terminava com algo executando contra o sistema real, o que evitou descobrir problemas só no fim.
- Contrato de API escrito antes do código: app, painel e backend evoluíram sem retrabalho de integração.
- Testes de integração contra o Oracle de verdade: pegaram erros que um banco simulado não pegaria, como a compilação de pacotes e as recusas das triggers.
- Automatizar cedo: o mesmo script instala o banco no computador, na FIAP e no pipeline, então o ambiente de teste e o de entrega não divergem.
- Documentação gerada por scripts a partir de evidências reais (saídas de terminal, relatórios de teste), e não digitada à mão.

O que não funcionou tão bem

- Trabalhar sozinho concentra todo o risco em uma pessoa e elimina a revisão por pares, que seria a defesa mais barata contra erros de modelagem.
- Não houve teste com usuários reais, então as hipóteses de valor (o morador usaria, o dono confiaria) seguem como hipóteses.
- Não existia orçamento nem registro de horas desde o início, o que obrigou a estimar custo ao final.

Aprendizados para projetos futuros

- Definir o orçamento e registrar horas desde a primeira semana, mesmo em projeto acadêmico.
- Colocar as regras de negócio onde os dados estão, e a IA apenas onde a incerteza é aceitável.
- Escrever o pipeline junto com o primeiro commit, não no final.
- Tratar a limpeza de dados de teste como código a ser revisado, com filtro explícito, nunca como um comando amplo.
- Validar com usuários reais antes de investir em escala.

5. Conclusão

O Smart HAS saiu de um protótipo com dados simulados para um sistema de três camadas com banco Oracle, regras de negócio em pacotes PL/SQL, auditoria por triggers, assistente de IA e entrega automatizada. Os objetivos propostos para este projeto foram atingidos: o ciclo completo de publicar, encontrar, pedir e devolver funciona de ponta a ponta, o painel acompanha atrasos e alertas, a IA está integrada ao aplicativo, e cada alteração é testada e publicada por um pipeline reprodutível.

Viabilidade

Tecnicamente, a solução é viável: usa tecnologias maduras (Java, Spring, Oracle, React Native, Angular), tem cobertura de testes automatizados e roda em contêiner. A principal incerteza não é de engenharia, e sim de adoção: depende de moradores dispostos a emprestar e de confiança entre vizinhos, hipóteses que só um piloto com usuários reais pode confirmar.

Impacto financeiro e operacional

Para o morador, cada empréstimo evita uma compra pontual. Como exemplo ilustrativo, e não como medição: se 500 empréstimos por ano evitassem compras de itens de R$ 120 em média, a economia circulante seria de R$ 60.000 por ano em uma comunidade. Para a operação, as regras no banco e a auditoria reduzem o trabalho de suporte, porque o painel mostra o que está atrasado e a trilha explica quem fez o quê. O custo estimado de construção, de R$ 61.464 no primeiro ano, é pequeno diante do alcance que uma varejista de reforma teria ao oferecer a rede a seus clientes.

Potencial de mercado e integração de IA

O mercado de reforma e construção residencial é recorrente, e o uso pontual de ferramentas é um ponto de atrito conhecido. A IA do projeto já cumpre um papel útil (anúncios completos e dúvidas respondidas), e o caminho natural é a recomendação por projeto, ligando o que existe na vizinhança ao catálogo da parceira. A arquitetura permite essa evolução sem mudar a base: a IA continua como camada de sugestão e a regra de negócio continua no banco.

Importância para a empresa parceira

Para a Leroy Merlin, a extensão posiciona a marca no começo da jornada de reforma, quando o cliente decide se vale a pena começar. Reduzir a barreira de custo das ferramentas, reforçar a agenda de sustentabilidade e gerar dados de demanda local por categoria são ganhos que um canal tradicional de vendas não entrega sozinho.

Como resultado final, o projeto entrega um protótipo funcional, testado, documentado e com código aberto, e um caminho claro de evolução para um piloto real.

Anexo A - Documentação e código-fonte

O relatório é acompanhado da documentação completa e do código-fonte no repositório do GitHub, que contém tudo o que é necessário para executar o projeto:

Pasta ou arquivo	Conteúdo
mobile-react-native/SmartHASApp	Aplicativo React Native (10 telas, mapa, ConectaIA)
backend/smarthas-api	API Spring Boot, testes de integração e Dockerfile
web/smarthas-admin	Painel administrativo em Angular
database	Scripts Oracle 01 a 08 e 99 (tabelas, carga, pacotes, triggers, consultas), executar_scripts.py, demonstrações e README com todos os objetos documentados
.github/workflows/ci-cd.yml	Pipeline de CI/CD
scripts/ci	Teste de fumaça, resumo de testes e dependências do pipeline
docs	Relatórios, slides e roteiros das fases 5, 6 e 7 e deste relatório, e os scripts que os geram
API_CONTRACT.md e README.md	Contrato da API e instruções de execução
iniciar_projeto.bat e docker-compose.yml	Subida local do conjunto

Quadro 10 - Estrutura do repositório

Como executar

1. Configurar as credenciais do Oracle em variáveis de ambiente (ORACLE_USER, ORACLE_PASSWORD, ORACLE_DSN) ou no arquivo de propriedades do backend, que não é versionado.

2. Instalar o schema: python database/executar_scripts.py (o --recriar apaga e instala de novo).

3. Rodar o backend: ./mvnw spring-boot:run em backend/smarthas-api, e os testes com SMARTHAS_DB_TESTS=true ./mvnw verify.

4. Rodar o painel (npm start em web/smarthas-admin) e o aplicativo (npx expo start em mobile-react-native/SmartHASApp). Em Windows, iniciar_projeto.bat sobe os serviços de uma vez.

Links

- Repositório: https://github.com/YuriRASousa/ComunidadeConectada_FASE5
- Pipeline (GitHub Actions): https://github.com/YuriRASousa/ComunidadeConectada_FASE5/actions/workflows/ci-cd.yml
- Vídeo do pitch do Enterprise Challenge (Atividade 4): https://youtu.be/egOV6KTYLeA
- Vídeo da Fase 7 com a demonstração do sistema: https://youtu.be/kXHb9uTTgTw

Referências

- BOTSMAN, R.; ROGERS, R. What's Mine Is Yours: The Rise of Collaborative Consumption. Harper Business, 2010.
- ELLEN MACARTHUR FOUNDATION. Towards the Circular Economy. 2013.
- HUMBLE, J.; FARLEY, D. Continuous Delivery. Addison-Wesley, 2010.
- CABINET OFFICE, GOVERNMENT OF JAPAN. Society 5.0, 5th Science and Technology Basic Plan. 2016.
- APINFO. Pesquisa Salarial 2025, rendimento mensal de cargos de TI no Brasil (coleta de agosto a setembro de 2025).
- ORACLE. Database PL/SQL Language Reference (pacotes, triggers e SQL dinâmico).